ESE 101 Homework 1

Tia Abraham

Octoboer 14, 2025

---

1 Low-e glazing

When we were remodeling Linde Laboratory, we wanted it to become energy efficient but were constrained by the mandate to preserve its historical appearance. For example, we could not change the appearance of the windows. However, the window glazing in the historical windows is new.

Part (a)

We decided double glazing (two glass panes with an air gap in between) to reduce heat conduction was not going to change the energy efficiency of the building much. Can you think of reasons why? (Strong hint: look at the windows and think about what the frames are made of.)

Even though double glazing is great for reducing heat transfer by conduction as we discussed in class, there are a couple of reasons why it might not have been a game-changer for the building's energy efficiency:

1. **Thermal Bridging through Frames:** The window frames of Linde Laboratory look like they're made of a thin metal (possibly steel or bronze), which means that they are most likely excellent conductors of heat. This creates a "thermal bridge" where heat can easily bypass the insulated glass panes. You could have the most efficient glass in the world, but if it's sitting in a highly conductive frame, the entire window unit will still leak a lot of heat.
2. **Radiative Heat Transfer:** Double glazing's main job is to stop conduction with an air gap, but it doesn't stop heat from radiating across that gap. As **section 2.4.3** of the *Physics of Earth's Climate* mentions, standard glass has a high absorptivity and emissivity for thermal infrared (radiant heat). The warm inner pane of glass radiates heat to the colder outer pane, which then radiates that heat to the outside. This leads to "radiant heat transfer from the warmer to the colder side even if the window is insulated against thermal conduction."

So, between the heat loss through the frames and the heat transfer from radiation, simply installing double-pane glass wouldn't have been enough to make the building significantly more energy-efficient.

Part (b)

In your own words, define albedo and emissivity, and explain their relationship. Be precise in your language!

Albedo is a measure of how much solar radiation a body reflects back into space. It's a fraction, so an albedo of 0 means the body absorbs all incoming sunlight, and an albedo of 1 means it's a perfect reflector.

Emissivity measures how well a body radiates thermal energy compared to a perfect blackbody at the same temperature. A blackbody is a perfect emitter and has an emissivity of 1. Most things aren't perfect emitters, so they have an emissivity less than 1.

Their relationship is described by Kirchhoff's Law, which states that for a body in thermodynamic equilibrium, its emissivity at a specific wavelength is equal to its absorptivity at that same wavelength ($a_\lambda = \epsilon_\lambda$).

This means that a good absorber is a good emitter, and a poor absorber is a poor emitter. So, a body that is good at absorbing thermal radiation is also good at radiating it away.

Part (c)

Regular glass has an emissivity of about 0.9 for thermal infrared radiation. If the outside temperature is 310 K, what is the energy flux with which the windows radiate into the building? What is the wavelength of maximum emission?

To find the energy flux radiated by the windows into the building, we use the Stefan-Boltzmann law, which describes the radiative energy flux from a body that is not a perfect blackbody:

$$\begin{aligned} F &= \epsilon\sigma T^4 && \textbf{[Stefan-Boltzmann Law, Eq. (2.13)]} \\ &= (0.9)\cdot(5.670\times10^{-8}\,\mathrm{W\,m^{-2}K^{-4}})\cdot(310\,\mathrm{K})^4 && \textbf{[Plug in values for } \epsilon, \sigma, \textbf{ and } T\textbf{]} \\ &= 471.3\,\mathrm{W\,m^{-2}} \end{aligned}$$

Thus, the energy flux with which the windows radiate into the building is $\boxed{471.3\ \mathrm{Wm^{-2}}}$. To find the wavelength of maximum emission, we use Wien's displacement law, which relates the peak emission wavelength of a body to its temperature:

$$\begin{aligned} \lambda_{\max} &= \frac{b}{T} && \textbf{[Wien's Displacement Law, Eq. (2.8)]} \\ &= \frac{2.9\times10^{-3}\,\mathrm{m\,K}}{310\,\mathrm{K}} && \textbf{[Plug in values for } b \textbf{ and } T\textbf{]} \\ &= 9.35\times10^{-6}\,\mathrm{m} && \textbf{[Calculate result in meters]} \\ &= 9.35\,\mu\mathrm{m} && \textbf{[Convert to micrometers]} \end{aligned}$$

Thus, the wavelength of maximum emission is $\boxed{9.35\ \mu\mathrm{m}}$.

Part (d)

Using glass with a low-emissivity ("low-e") metal oxide coating was found to lead to substantial energy savings. The coating reduces the thermal infrared emissivity of the glass, to a value around 0.04. Qualitatively describe why this reduction in emissivity reduces the building's energy demand.

Low-e coatings save energy by controlling radiant heat by:

1. **Reflecting Heat:** According to Kirchhoff's Law, a material with low emissivity is also a poor absorber of thermal energy, as we described in **part (b)**. Instead of absorbing radiant heat, the low-e coating reflects it. This means in the winter it keeps the building's heat inside, and in the summer it reflects the sun's heat back outside.
2. **Radiating Very Little Heat:** The Stefan-Boltzmann Law shows that the amount of heat a surface radiates is directly proportional to its emissivity ($F = \epsilon\sigma T^4$); so, since the low-e coating has a lower emissivity (around 0.04), the glass itself radiates less heat, which helps keep the indoor temperature stable year-round.

By doing both, the coating drastically reduces the amount of energy that buildings' heating and A/C systems need to use.

2 Root-Mean-Square Velocity

Question 2

Calculate the root-mean-square (rms) velocity of Nitrogen gas and Oxygen gas at 300K, 275K, and 250K.

The root-mean-square velocity is given by the equation $v_{rms} = \sqrt{\frac{3R_0T}{m_a}}$ (from **section 2.4.1**), where R_0 is the universal gas constant ($8.315\,\text{J K}^{-1}\text{mol}^{-1}$), T is the temperature in Kelvin, and m_a is the molar mass in kg/mol (the textbook used the molar mass of air, but here we'll use the molar mass of nitrogen and oxygen gas respectively).

Nitrogen Gas (N_2)

The molar mass of molecular nitrogen (N_2) is $28\,\text{g mol}^{-1}$ or $0.028\,\text{kg mol}^{-1}$. Then, the rms values at each temperature is:

$$
\begin{aligned}
v^N_{300\text{K}} &= \sqrt{\frac{3R_0T}{m_a}} && \textbf{[rms velocity equation]} \\
&= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(300\,\text{K})}{0.028\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 517.0\,\text{m s}^{-1} \\
v^N_{275\text{K}} &= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(275\,\text{K})}{0.028\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 495.0\,\text{m s}^{-1} \\
v^N_{250\text{K}} &= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(250\,\text{K})}{0.028\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 471.9\,\text{m s}^{-1}
\end{aligned}
$$

Oxygen Gas (O_2)

The molar mass of molecular oxygen (O_2) is $32\,\text{g mol}^{-1}$ or $0.032\,\text{kg mol}^{-1}$.

$$
\begin{aligned}
v^O_{300\text{K}} &= \sqrt{\frac{3R_0T}{m_a}} && \textbf{[rms velocity equation]} \\
&= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(300\,\text{K})}{0.032\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 483.6\,\text{m s}^{-1} \\
v^O_{275\text{K}} &= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(275\,\text{K})}{0.032\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 463.0\,\text{m s}^{-1} \\
v^O_{250\text{K}} &= \sqrt{\frac{3\cdot(8.315\,\text{J K}^{-1}\text{mol}^{-1})\cdot(250\,\text{K})}{0.032\,\text{kg mol}^{-1}}} && \textbf{[Plug in values]} \\
&= 441.5\,\text{m s}^{-1}
\end{aligned}
$$

ESE 101: Programming Module 1

Phenomena: Observed features of the atmospheric circulation

Ronak Patel, Ryan Ward, Jordan Benjamin, Shirui Peng, Dave Bonan, Tapio Schneider

In this module, you will learn how to read in an plot data of the atmosphere in **python**. We will use output from a state-of-the-art climate model (CESM2).

You will need to have software installed to open, read, and run Jupyter notebooks such as this one. The best way to do this is to install Anaconda from: https://docs.anaconda.com/anaconda/install/.
This should install Jupyter's notebook software (e.g. this document) and JupyterLab software (a more complete IDE), but, should you need to install them individually, you can via https://jupyter.org/install.
You can choose to intstall jupyterlab `conda install -c conda-forge jupyterlab`, or just the notebook `conda install -c conda-forge notebook`.

An anaconda installation should have installed many the packages you need, the full required list is `matplotlib`, `cartopy`, `numpy`, `math`, `netcdf4`, `xarray`, `nc-time-axis`. Most of these are fairly common in python scientific computing, and `nc-time-axis` handles the time format conventions used in modern climate model datasets.

We'll create a specific python environment for this homework set -- a good practice to make sure all these tools we are using remain compatible and don't conflict with other projects and work we might need later.

Setup python environment to run this HW in:

The cells below describe a process to set up a python environment to run the code for this HW in.
The lines can be run in the notebook with the prepended exclamation `!` marks, or in terminal/anaconda terminal without them.

In [1]:
```
# install these packages locally so you can access the environment we're abo
!conda install -c conda-forge --yes ipykernel nb_conda_kernels nb_conda
```

```
2 channel Terms of Service accepted
Channels:
 - conda-forge
 - defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done


==> WARNING: A newer version of conda exists. <==
    current version: 25.7.0
    latest version: 25.9.1

Please update conda by running

    $ conda update -n base -c conda-forge conda



# All requested packages already installed.
```

> Completely close and restart jupyter/jupyterlab for these changes take effect! Then reopen and continue below

Now let's create our homework environment, which we'll call `hw1_ese101` and install our necessary packages in it

In [2]:
```
# create our homework environment
!conda create -n hw1_ese101 python=3.13 --yes
# install packages in this new enviornment
!conda install -c conda-forge --yes -n hw1_ese101 matplotlib cartopy numpy r
```

```
2 channel Terms of Service accepted
Channels:
 - defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /opt/anaconda3/envs/hw1_ese101

  added / updated specs:
    - python=3.13


The following NEW packages will be INSTALLED:

  bzip2              pkgs/main/osx-arm64::bzip2-1.0.8-h80987f9_6
  ca-certificates    pkgs/main/osx-arm64::ca-certificates-2025.9.9-hca03da5_
0
  expat              pkgs/main/osx-arm64::expat-2.7.1-h313beb8_0
  libcxx             pkgs/main/osx-arm64::libcxx-20.1.8-h8869778_0
  libffi             pkgs/main/osx-arm64::libffi-3.4.4-hca03da5_1
  libmpdec           pkgs/main/osx-arm64::libmpdec-4.0.0-h80987f9_0
  libzlib            pkgs/main/osx-arm64::libzlib-1.3.1-h5f15de7_0
  ncurses            pkgs/main/osx-arm64::ncurses-6.5-hee39554_0
  openssl            pkgs/main/osx-arm64::openssl-3.0.18-h9b4081a_0
  pip                pkgs/main/noarch::pip-25.2-pyhc872135_1
  python             pkgs/main/osx-arm64::python-3.13.7-hc28b8d9_100_cp313
  python_abi         pkgs/main/osx-arm64::python_abi-3.13-1_cp313
  readline           pkgs/main/osx-arm64::readline-8.3-h0b18652_0
  setuptools         pkgs/main/osx-arm64::setuptools-80.9.0-py313hca03da5_0
  sqlite             pkgs/main/osx-arm64::sqlite-3.50.2-h79febb2_1
  tk                 pkgs/main/osx-arm64::tk-8.6.15-hcd8a7d5_0
  tzdata             pkgs/main/noarch::tzdata-2025b-h04d1e81_0
  wheel              pkgs/main/osx-arm64::wheel-0.45.1-py313hca03da5_0
  xz                 pkgs/main/osx-arm64::xz-5.6.4-h80987f9_1
  zlib               pkgs/main/osx-arm64::zlib-1.3.1-h5f15de7_0



Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate hw1_ese101
#
# To deactivate an active environment, use
#
#     $ conda deactivate

2 channel Terms of Service accepted
```

```
Channels:
 - conda-forge
 - defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done


==> WARNING: A newer version of conda exists. <==
    current version: 25.7.0
    latest version: 25.9.1

Please update conda by running

    $ conda update -n base -c conda-forge conda



## Package Plan ##

  environment location: /opt/anaconda3/envs/hw1_ese101

  added / updated specs:
    - cartopy
    - ipykernel
    - ipython
    - matplotlib
    - metpy
    - nb_conda
    - nb_conda_kernels
    - nc-time-axis
    - netcdf4
    - numpy
    - xarray


The following NEW packages will be INSTALLED:

  _python_abi3_supp~ conda-forge/noarch::_python_abi3_support-1.0-hd8ed1ab_2
  anyio              conda-forge/noarch::anyio-4.11.0-pyhcf101f3_0
  appnope            conda-forge/noarch::appnope-0.1.4-pyhd8ed1ab_1
  argon2-cffi        conda-forge/noarch::argon2-cffi-25.1.0-pyhd8ed1ab_0
  argon2-cffi-bindi~ conda-forge/osx-arm64::argon2-cffi-bindings-25.1.0-py31
3h6535dbc_1
  arrow              conda-forge/noarch::arrow-1.3.0-pyhd8ed1ab_1
  asttokens          conda-forge/noarch::asttokens-3.0.0-pyhd8ed1ab_1
  attrs              conda-forge/noarch::attrs-25.4.0-pyh71513ae_0
  beautifulsoup4     conda-forge/noarch::beautifulsoup4-4.14.2-pyha770c72_0
  bleach             conda-forge/noarch::bleach-6.2.0-pyh29332c3_4
  bleach-with-css    conda-forge/noarch::bleach-with-css-6.2.0-h82add2a_4
  blosc              conda-forge/osx-arm64::blosc-1.21.6-h7dd00d9_1
  brotli             conda-forge/osx-arm64::brotli-1.1.0-h6caf38d_4
  brotli-bin         conda-forge/osx-arm64::brotli-bin-1.1.0-h6caf38d_4
  brotli-python      conda-forge/osx-arm64::brotli-python-1.1.0-py313hb4b787
7_4
  c-ares             conda-forge/osx-arm64::c-ares-1.34.5-h5505292_0
```

```
  cached-property    conda-forge/noarch::cached-property-1.5.2-hd8ed1ab_1
  cached_property    conda-forge/noarch::cached_property-1.5.2-pyha770c72_1
  cartopy            conda-forge/osx-arm64::cartopy-0.25.0-py313h7d16b84_1
  certifi            conda-forge/noarch::certifi-2025.10.5-pyhd8ed1ab_0
  cffi               conda-forge/osx-arm64::cffi-1.17.1-py313hc845a76_0
  cftime             conda-forge/osx-arm64::cftime-1.6.4-py313h2732efb_2
  charset-normalizer conda-forge/noarch::charset-normalizer-3.4.4-pyhd8ed1ab
_0
  comm               conda-forge/noarch::comm-0.2.3-pyhe01879c_0
  contourpy          conda-forge/osx-arm64::contourpy-1.3.3-py313hc50a443_2
  cpython            conda-forge/noarch::cpython-3.13.8-py313hd8ed1ab_101
  cycler             conda-forge/noarch::cycler-0.12.1-pyhd8ed1ab_1
  debugpy            conda-forge/osx-arm64::debugpy-1.8.17-py313hc37fe24_0
  decorator          conda-forge/noarch::decorator-5.2.1-pyhd8ed1ab_0
  defusedxml         conda-forge/noarch::defusedxml-0.7.1-pyhd8ed1ab_0
  exceptiongroup     conda-forge/noarch::exceptiongroup-1.3.0-pyhd8ed1ab_0
  executing          conda-forge/noarch::executing-2.2.1-pyhd8ed1ab_0
  flexcache          conda-forge/noarch::flexcache-0.3-pyhd8ed1ab_1
  flexparser         conda-forge/noarch::flexparser-0.4-pyhd8ed1ab_1
  fonttools          conda-forge/osx-arm64::fonttools-4.60.1-py313h7d74516_0
  fqdn               conda-forge/noarch::fqdn-1.5.1-pyhd8ed1ab_1
  freetype           conda-forge/osx-arm64::freetype-2.14.1-hce30654_0
  geos               conda-forge/osx-arm64::geos-3.14.0-h4bcf65f_0
  h2                 conda-forge/noarch::h2-4.3.0-pyhcf101f3_0
  hdf4               conda-forge/osx-arm64::hdf4-4.2.15-h2ee6834_7
  hdf5               conda-forge/osx-arm64::hdf5-1.14.6-nompi_he65715a_103
  hpack              conda-forge/noarch::hpack-4.1.0-pyhd8ed1ab_0
  hyperframe         conda-forge/noarch::hyperframe-6.1.0-pyhd8ed1ab_0
  icu                conda-forge/osx-arm64::icu-73.2-hc8870d7_0
  idna               conda-forge/noarch::idna-3.11-pyhd8ed1ab_0
  importlib-metadata conda-forge/noarch::importlib-metadata-8.7.0-pyhe01879c
_1
  ipykernel          conda-forge/noarch::ipykernel-7.0.1-pyh5552912_0
  ipython            conda-forge/noarch::ipython-8.21.0-pyh707e725_0
  ipython_genutils   conda-forge/noarch::ipython_genutils-0.2.0-pyhd8ed1ab_2
  isoduration        conda-forge/noarch::isoduration-20.11.0-pyhd8ed1ab_1
  jedi               conda-forge/noarch::jedi-0.19.2-pyhd8ed1ab_1
  jinja2             conda-forge/noarch::jinja2-3.1.6-pyhd8ed1ab_0
  jsonpointer        conda-forge/osx-arm64::jsonpointer-3.0.0-py313h8f79df9_
2
  jsonschema         conda-forge/noarch::jsonschema-4.25.1-pyhe01879c_0
  jsonschema-specif~ conda-forge/noarch::jsonschema-specifications-2025.9.1-
pyhcf101f3_0
  jsonschema-with-f~ conda-forge/noarch::jsonschema-with-format-nongpl-4.25.
1-he01879c_0
  jupyter_client     conda-forge/noarch::jupyter_client-8.6.3-pyhd8ed1ab_1
  jupyter_core       conda-forge/noarch::jupyter_core-5.8.1-pyh31011fe_0
  jupyter_events     conda-forge/noarch::jupyter_events-0.12.0-pyh29332c3_0
  jupyter_server     conda-forge/noarch::jupyter_server-2.17.0-pyhcf101f3_0
  jupyter_server_te~ conda-forge/noarch::jupyter_server_terminals-0.5.3-pyhd
8ed1ab_1
  jupyterlab_pygmen~ conda-forge/noarch::jupyterlab_pygments-0.3.0-pyhd8ed1a
b_2
  kiwisolver         conda-forge/osx-arm64::kiwisolver-1.4.9-py313hf88c9ab_1
  krb5               conda-forge/osx-arm64::krb5-1.21.3-h237132a_0
  lark               conda-forge/noarch::lark-1.3.0-pyhd8ed1ab_0
```

```
  lcms2              conda-forge/osx-arm64::lcms2-2.17-h7eeda09_0
  lerc               conda-forge/osx-arm64::lerc-4.0.0-hd64df32_1
  libaec             conda-forge/osx-arm64::libaec-1.1.4-h51d1e36_0
  libblas            conda-forge/osx-arm64::libblas-3.9.0-37_h51639a9_openbl
as
  libbrotlicommon    conda-forge/osx-arm64::libbrotlicommon-1.1.0-h6caf38d_4
  libbrotlidec       conda-forge/osx-arm64::libbrotlidec-1.1.0-h6caf38d_4
  libbrotlienc       conda-forge/osx-arm64::libbrotlienc-1.1.0-h6caf38d_4
  libcblas           conda-forge/osx-arm64::libcblas-3.9.0-37_hb0561ab_openb
las
  libcurl            conda-forge/osx-arm64::libcurl-8.14.1-h73640d1_0
  libdeflate         conda-forge/osx-arm64::libdeflate-1.22-hd74edd7_0
  libedit            conda-forge/osx-arm64::libedit-3.1.20250104-pl5321hafb1
f1b_0
  libev              conda-forge/osx-arm64::libev-4.33-h93a5062_2
  libfreetype        conda-forge/osx-arm64::libfreetype-2.14.1-hce30654_0
  libfreetype6       conda-forge/osx-arm64::libfreetype6-2.14.1-h6da58f4_0
  libgfortran        conda-forge/osx-arm64::libgfortran-15.2.0-hfcf01ff_1
  libgfortran5       conda-forge/osx-arm64::libgfortran5-15.2.0-h742603c_1
  libiconv           conda-forge/osx-arm64::libiconv-1.18-h23cfdf5_2
  libjpeg-turbo      conda-forge/osx-arm64::libjpeg-turbo-3.1.0-h5505292_0
  liblapack          conda-forge/osx-arm64::liblapack-3.9.0-37_hd9741b5_open
blas
  libnetcdf          conda-forge/osx-arm64::libnetcdf-4.9.3-nompi_h6808abe_1
02
  libnghttp2         conda-forge/osx-arm64::libnghttp2-1.67.0-hc438710_0
  libopenblas        conda-forge/osx-arm64::libopenblas-0.3.30-openmp_h60d53
f8_2
  libpng             conda-forge/osx-arm64::libpng-1.6.50-h280e0eb_1
  libsodium          conda-forge/osx-arm64::libsodium-1.0.20-h99b78c6_0
  libsqlite          conda-forge/osx-arm64::libsqlite-3.50.4-hf8de324_0
  libssh2            conda-forge/osx-arm64::libssh2-1.11.1-h1590b86_0
  libtiff            conda-forge/osx-arm64::libtiff-4.7.0-hfce79cd_1
  libwebp-base       conda-forge/osx-arm64::libwebp-base-1.6.0-h07db88b_0
  libxcb             conda-forge/osx-arm64::libxcb-1.17.0-hdb1d25a_0
  libxml2            pkgs/main/osx-arm64::libxml2-2.13.8-h0b34f26_0
  libzip             conda-forge/osx-arm64::libzip-1.11.2-h1336266_0
  llvm-openmp        conda-forge/osx-arm64::llvm-openmp-20.1.8-h4a912ad_3
  lz4-c              conda-forge/osx-arm64::lz4-c-1.10.0-h286801f_1
  markupsafe         conda-forge/osx-arm64::markupsafe-3.0.3-py313h7d74516_0
  matplotlib         conda-forge/osx-arm64::matplotlib-3.10.7-py313h39782a4_
0
  matplotlib-base    conda-forge/osx-arm64::matplotlib-base-3.10.7-py313h580
42b9_0
  matplotlib-inline  conda-forge/noarch::matplotlib-inline-0.1.7-pyhd8ed1ab_
1
  metpy              conda-forge/noarch::metpy-1.7.1-pyhd8ed1ab_0
  mistune            conda-forge/noarch::mistune-3.1.4-pyhcf101f3_0
  munkres            conda-forge/noarch::munkres-1.1.4-pyhd8ed1ab_1
  nb_conda           conda-forge/noarch::nb_conda-2.2.1-pyh707e725_7
  nb_conda_kernels   conda-forge/noarch::nb_conda_kernels-2.5.1-pyh707e725_2
  nbclassic          conda-forge/noarch::nbclassic-1.3.3-pyhcf101f3_0
  nbclient           conda-forge/noarch::nbclient-0.10.2-pyhd8ed1ab_0
  nbconvert-core     conda-forge/noarch::nbconvert-core-7.16.6-pyh29332c3_0
  nbformat           conda-forge/noarch::nbformat-5.10.4-pyhd8ed1ab_1
  nc-time-axis       conda-forge/noarch::nc-time-axis-1.4.1-pyhd8ed1ab_1
```

```
  nest-asyncio       conda-forge/noarch::nest-asyncio-1.6.0-pyhd8ed1ab_1
  netcdf4            conda-forge/osx-arm64::netcdf4-1.7.3-nompi_py313hb28a5c
b_100
  notebook           conda-forge/noarch::notebook-6.5.4-pyha770c72_0
  notebook-shim      conda-forge/noarch::notebook-shim-0.2.4-pyhd8ed1ab_1
  numpy              conda-forge/osx-arm64::numpy-2.3.3-py313h9771d21_0
  openjpeg           conda-forge/osx-arm64::openjpeg-2.5.3-h889cd5d_1
  overrides          conda-forge/noarch::overrides-7.7.0-pyhd8ed1ab_1
  packaging          conda-forge/noarch::packaging-25.0-pyh29332c3_1
  pandas             conda-forge/osx-arm64::pandas-2.3.3-py313h7d16b84_1
  pandocfilters      conda-forge/noarch::pandocfilters-1.5.0-pyhd8ed1ab_0
  parso              conda-forge/noarch::parso-0.8.5-pyhcf101f3_0
  pexpect            conda-forge/noarch::pexpect-4.9.0-pyhd8ed1ab_1
  pickleshare        conda-forge/noarch::pickleshare-0.7.5-pyhd8ed1ab_1004
  pillow             conda-forge/osx-arm64::pillow-11.3.0-py313he4c6d0d_3
  pint               conda-forge/noarch::pint-0.25-pyhe01879c_0
  platformdirs       conda-forge/noarch::platformdirs-4.5.0-pyhcf101f3_0
  pooch              conda-forge/noarch::pooch-1.8.2-pyhd8ed1ab_3
  proj               conda-forge/osx-arm64::proj-9.7.0-hf83150c_0
  prometheus_client  conda-forge/noarch::prometheus_client-0.23.1-pyhd8ed1ab
_0
  prompt-toolkit     conda-forge/noarch::prompt-toolkit-3.0.52-pyha770c72_0
  psutil             conda-forge/osx-arm64::psutil-7.1.0-py313h6535dbc_0
  pthread-stubs      conda-forge/osx-arm64::pthread-stubs-0.4-hd74edd7_1002
  ptyprocess         conda-forge/noarch::ptyprocess-0.7.0-pyhd8ed1ab_1
  pure_eval          conda-forge/noarch::pure_eval-0.2.3-pyhd8ed1ab_1
  pycparser          conda-forge/noarch::pycparser-2.22-pyh29332c3_1
  pygments           conda-forge/noarch::pygments-2.19.2-pyhd8ed1ab_0
  pyobjc-core        conda-forge/osx-arm64::pyobjc-core-11.0-py313hb6afeec_0
  pyobjc-framework-~ conda-forge/osx-arm64::pyobjc-framework-cocoa-11.0-py31
3hb6afeec_0
  pyparsing          conda-forge/noarch::pyparsing-3.2.5-pyhcf101f3_0
  pyproj             conda-forge/osx-arm64::pyproj-3.7.2-py313h698103d_2
  pyshp              conda-forge/noarch::pyshp-3.0.2-pyhd8ed1ab_0
  pysocks            conda-forge/noarch::pysocks-1.7.1-pyha55dd90_7
  python-dateutil    conda-forge/noarch::python-dateutil-2.9.0.post0-pyhe018
79c_2
  python-fastjsonsc~ conda-forge/noarch::python-fastjsonschema-2.21.2-pyhe01
879c_0
  python-gil         conda-forge/noarch::python-gil-3.13.8-h4df99d1_101
  python-json-logger conda-forge/noarch::python-json-logger-2.0.7-pyhd8ed1ab
_0
  python-tzdata      conda-forge/noarch::python-tzdata-2025.2-pyhd8ed1ab_0
  pytz               conda-forge/noarch::pytz-2025.2-pyhd8ed1ab_0
  pyyaml             conda-forge/osx-arm64::pyyaml-6.0.3-py313h7d74516_0
  pyzmq              conda-forge/osx-arm64::pyzmq-27.1.0-py312hd65ceae_0
  qhull              conda-forge/osx-arm64::qhull-2020.2-h420ef59_5
  referencing        conda-forge/noarch::referencing-0.37.0-pyhcf101f3_0
  requests           conda-forge/noarch::requests-2.32.5-pyhd8ed1ab_0
  rfc3339-validator  conda-forge/noarch::rfc3339-validator-0.1.4-pyhd8ed1ab_
1
  rfc3986-validator  conda-forge/noarch::rfc3986-validator-0.1.1-pyh9f0ad1d_
0
  rfc3987-syntax     conda-forge/noarch::rfc3987-syntax-1.1.0-pyhe01879c_1
  rpds-py            conda-forge/osx-arm64::rpds-py-0.27.1-py313h80e0809_1
  scipy              conda-forge/osx-arm64::scipy-1.16.2-py313h0d10b07_0
```

```
  send2trash         conda-forge/noarch::send2trash-1.8.3-pyh31c8845_1
  shapely            conda-forge/osx-arm64::shapely-2.1.2-py313hfb5a6ed_0
  six                conda-forge/noarch::six-1.17.0-pyhe01879c_1
  snappy             conda-forge/osx-arm64::snappy-1.2.2-hd121638_0
  sniffio            conda-forge/noarch::sniffio-1.3.1-pyhd8ed1ab_1
  soupsieve          conda-forge/noarch::soupsieve-2.8-pyhd8ed1ab_0
  stack_data         conda-forge/noarch::stack_data-0.6.3-pyhd8ed1ab_1
  terminado          conda-forge/noarch::terminado-0.18.1-pyh31c8845_0
  tinycss2           conda-forge/noarch::tinycss2-1.4.0-pyhd8ed1ab_0
  tornado            conda-forge/osx-arm64::tornado-6.5.2-py313hcdf3177_1
  traitlets          conda-forge/noarch::traitlets-5.9.0-pyhd8ed1ab_0
  types-python-date~ conda-forge/noarch::types-python-dateutil-2.9.0.2025100
8-pyhd8ed1ab_0
  typing-extensions  conda-forge/noarch::typing-extensions-4.15.0-h396c80c_0
  typing_extensions  conda-forge/noarch::typing_extensions-4.15.0-pyhcf101f3
_0
  typing_utils       conda-forge/noarch::typing_utils-0.1.0-pyhd8ed1ab_1
  uri-template       conda-forge/noarch::uri-template-1.3.0-pyhd8ed1ab_1
  urllib3            conda-forge/noarch::urllib3-2.5.0-pyhd8ed1ab_0
  wcwidth            conda-forge/noarch::wcwidth-0.2.14-pyhd8ed1ab_0
  webcolors          conda-forge/noarch::webcolors-24.11.1-pyhd8ed1ab_0
  webencodings       conda-forge/noarch::webencodings-0.5.1-pyhd8ed1ab_3
  websocket-client   conda-forge/noarch::websocket-client-1.9.0-pyhd8ed1ab_0
  xarray             conda-forge/noarch::xarray-2025.10.1-pyhd8ed1ab_0
  xorg-libxau        conda-forge/osx-arm64::xorg-libxau-1.0.12-h5505292_0
  xorg-libxdmcp      conda-forge/osx-arm64::xorg-libxdmcp-1.1.5-hd74edd7_0
  yaml               conda-forge/osx-arm64::yaml-0.2.5-h925e9cb_3
  zeromq             conda-forge/osx-arm64::zeromq-4.3.5-h888dc83_9
  zipp               conda-forge/noarch::zipp-3.23.0-pyhd8ed1ab_0
  zstandard          conda-forge/osx-arm64::zstandard-0.25.0-py313h9734d34_0
  zstd               conda-forge/osx-arm64::zstd-1.5.7-h6491c7d_2

The following packages will be UPDATED:

  ca-certificates    pkgs/main/osx-arm64::ca-certificates-~ --> conda-forge/
noarch::ca-certificates-2025.10.5-hbd8a1cb_0
  openssl              pkgs/main::openssl-3.0.18-h9b4081a_0 --> conda-forg
e::openssl-3.5.4-h5503f6c_0



Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: / Enabling nb_conda_kernels...
CONDA_PREFIX: /opt/anaconda3/envs/hw1_ese101
Status: enabled

| /opt/anaconda3/envs/hw1_ese101/lib/python3.13/site-packages/nb_conda/handl
ers.py:17: UserWarning: pkg_resources is deprecated as an API. See https://s
etuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is
slated for removal as early as 2025-11-30. Refrain from using this package o
r pin to Setuptools<81.
  from pkg_resources import parse_version
Enabling notebook extension nb_conda/main...
```

```
      - Validating: OK
Enabling tree extension nb_conda/tree...
      - Validating: OK
/opt/anaconda3/envs/hw1_ese101/lib/python3.13/site-packages/nb_conda/handler
s.py:17: UserWarning: pkg_resources is deprecated as an API. See https://set
uptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is s
lated for removal as early as 2025-11-30. Refrain from using this package or
pin to Setuptools<81.
  from pkg_resources import parse_version
Enabling: nb_conda
- Writing config: /opt/anaconda3/envs/hw1_ese101/etc/jupyter
    - Validating...
      nb_conda 2.2.1 OK

done
```

The `nb_conda_kernels` package should take care of initializing new kernels (python instances) for us in our new environment. We just need to select the correct kernel from the relevant dropdown menu, it should look something like `[conda env: hw1_ese101]`

Alternatively, if something goes wrong, you could also separately create the kernel by running these commands in your anaconda terminal:

`conda activate hw1_ese101` to switch to our new environment,
`!ipython kernel install --user --name=hw1_ese101` to start a new kernel there,
`conda deactivate` to switch back to our default environment

Then, the kernel in our homework environment should show up in the kernel menu as `hw1_ese101`, possible after needing a refresh.

In [3]:
```
# To see all the kernels you've created, run
!python -m nb_conda_kernels list
```

```
[ListKernelSpecs] nb_conda_kernels | enabled, 2 kernels found.
Available kernels:
  conda-base-py              /opt/anaconda3/share/jupyter/kernels/python3
  conda-env-hw1_ese101-py    /opt/anaconda3/envs/hw1_ese101/share/jupyter/ke
rnels/python3
```

A few final notes:

- **Run cells in this notebook by pressing `Ctrl+Enter`** (e.g., to fill in cells for answers marked `Answer:`)
- **Double click markdown (text) cells to edit them**

In this notebook, all the code is pre-prepared and should work without adjustment. Future notebooks will require you to, where prompted, write some simple code, so do your best to understand the examples we have provided here! And don't forget to reach out to the TAs with any and all questions!

Let's begin!!

Import python packages required for this notebook:

In [4]:
```
import matplotlib as mpl        # Python's default plotting package https://
import matplotlib.pyplot as plt
import cartopy.crs as ccrs      # Mapping  package https://scitools.org.uk/c
from   cartopy.mpl.gridliner import LONGITUDE_FORMATTER, LATITUDE_FORMATTER
import numpy as np              # Python's numerical and array calculations
import math                     # Python's default math package https://docs
import xarray as xr             # Python package for labeled datasets and ar
```

Define a function to plot data on a globe:

In [5]:
```
def cylindrical_equidistant_projection(lats,lons,data,ticks,c_ticks,cmap='Rc
    """
    A function to plot data on a globe
    """
    if fig is None:
        fig=plt.figure(figsize=(8, 5)) # initialize a figure

    lon2d, lat2d = np.meshgrid(lons, lats) # create 2D lat-lon grid
    if ax is None:
        ax = plt.axes(projection=ccrs.PlateCarree()) # create axes with assi

    ax.set_title(title,y=1.1) # add a plot title
    ax.set_global() # show the entire globe
    ax.coastlines() # plot coastlines
    gl = ax.gridlines(crs=ccrs.PlateCarree(), draw_labels=True,linewidth=0.7
    gl.bottom_labels = False # do not label longitudes on bottom
    gl.right_labels  = False  # do not lobal latitudes on right
    gl.xformatter = LONGITUDE_FORMATTER # format in longitude format
    gl.yformatter = LATITUDE_FORMATTER  # format in latitude format

    cs=ax.contourf(lons, lats, data,ticks,cmap=cmap) # plot data

    # create colorbar
    cax,kw = mpl.colorbar.make_axes(ax,location='bottom',pad=0.05,shrink=0.9
    cbar=fig.colorbar(cs,cax=cax,orientation='horizontal',ticks=c_ticks)
    cbar.set_label(c_label)
```

Now let's read monthly surface temperature from CESM2:

In [6]:
```
filepath = 'tas_Amon_CESM2_01_12_1981_2012.nc'  # choose this dataset, month
surface_temperature = xr.open_dataset(filepath) # open the file
surface_temperature                             # examine its data
```

Out[6]: xarray.Dataset

► Dimensions: (**lon**: 288, bnds: 2, **lat**: 192, **time**: 12)

▼ Coordinates:

lon	(lon)	float64	0.0 1.25 2.5 ... 356.2 357.5...
lat	(lat)	float64	-90.0 -89.06 -88.12 ... 89.0...
time	(time)	object	2010-01-15 12:00:00 ... 20...

▼ Data variables:

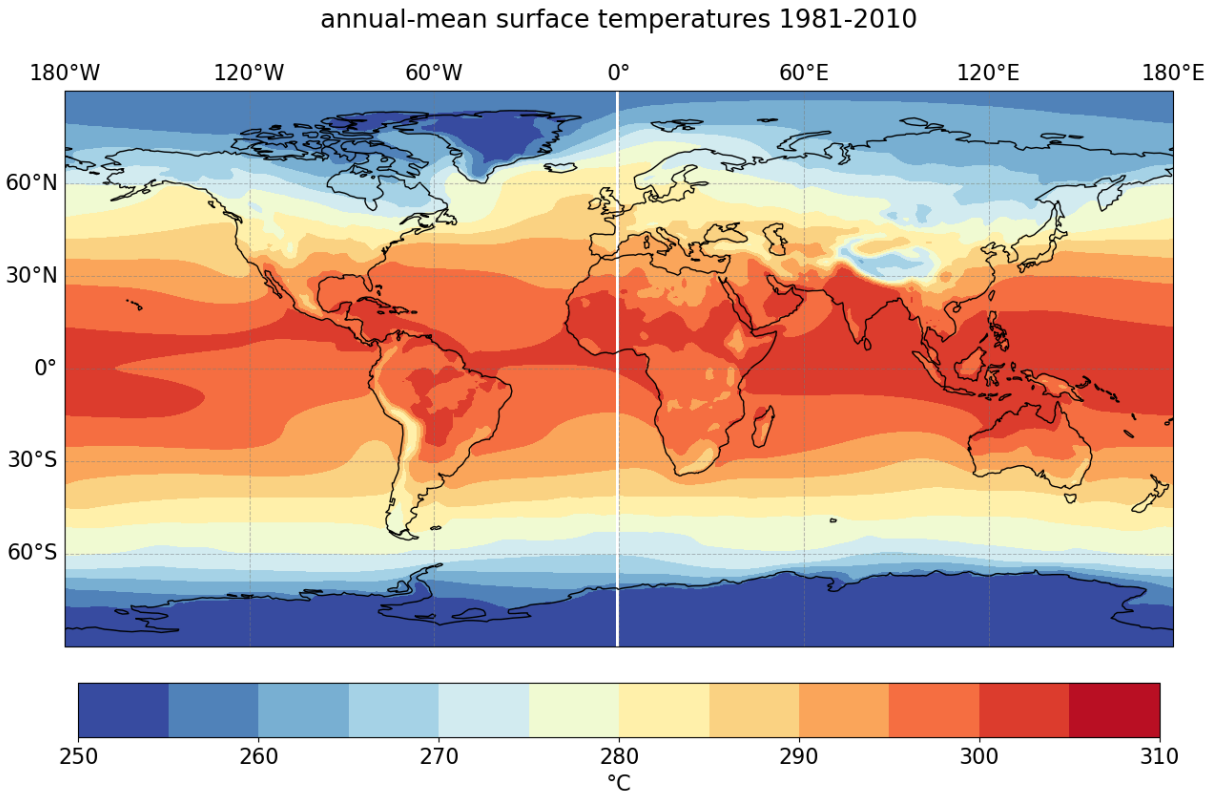
lon_bnds	(lon, bnds)	float64	...
lat_bnds	(lat, bnds)	float64	...
tas	(time, lat, lon)	float32	...

► Attributes: (44)

You'll note the data in `surface_temperature` is defined over several coordinates, `lat, lon, time` and the data variable we want is `tas`. We can access this variable with `surface_temperature['tas']`.

Now let's plot annual-mean surface temperature, and describe it:

In [7]:
```
plt.rcParams.update({'font.size': 16})  # change the font size
fig = plt.figure(figsize=(15,10))       # initialize a fairly large figure
ann_mean = surface_temperature['tas'].mean('time') # take the mean of our da
ann_mean = ann_mean.where(ann_mean > 250, 250) # replace values where ann_me
ann_mean = ann_mean.where(ann_mean < 310, 310) # replace values where ann_me
title = 'annual-mean surface temperatures 1981-2010'
cylindrical_equidistant_projection(surface_temperature['lat'],surface_temper
```



Decribe features you see in the above plot.

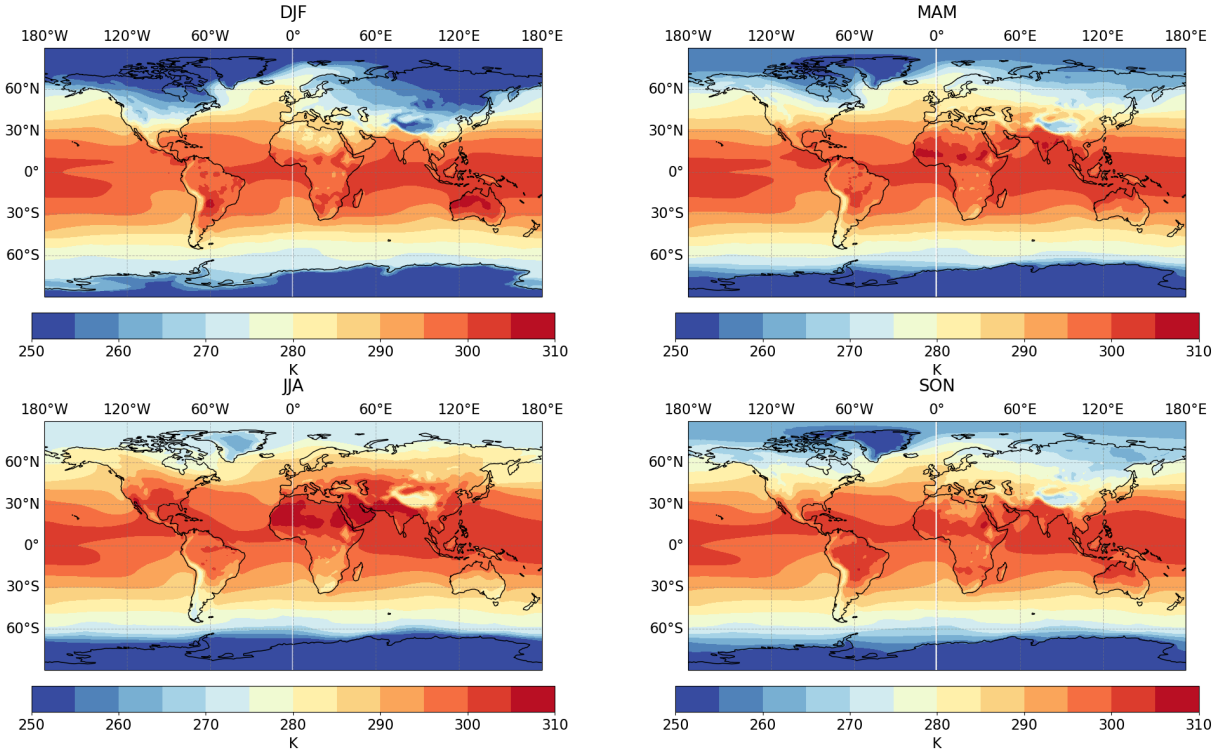
Answer:

- The most obvious pattern is that temperatures are highest near the equator and get colder toward the poles.
- The temperature is relatively uniform across roughly 30°N and 30°S. Outside this zone, the temperature drops off much more rapidly with latitude.
- High-altitude regions are noticeably colder than the areas around them. This is clear on the map with the Tibetan Plateau, the ice sheets of Greenland and Antarctica, and the Andes mountains.
- The regions poleward of about 60° latitude have annual-mean temperatures below the freezing point of water (273.15 K), which is consistent with the presence of permanent ice and permafrost.

Now let's take and plot some seasonal means:

In [8]:
```
plt.rcParams.update({'font.size': 16})  # change the font size
tas_seasonal = surface_temperature['tas'].groupby('time.season').mean() # gr
```

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(24,14), subplot_kw={'pro
for i, season in enumerate(('DJF', 'MAM', 'JJA', 'SON')):
    seas_data = tas_seasonal.sel(season=season) # select teh season we want
    seas_data = seas_data.where(seas_data > 250, 250) # replace values where
    seas_data = seas_data.where(seas_data < 310, 310) # replace values where
    cylindrical_equidistant_projection(surface_temperature['lat'],surface_te
```



Describe differences across each season.

Answer:

- The biggest temperature variations happen between the solstice seasons (DJF and JJA), especially in the middle and high latitudes.
    - In DJF (Northern Hemisphere winter), the northern continents, particularly Siberia and North America, get extremely cold, with average temperatures dropping to -40°C.
    - In JJA (Northern Hemisphere summer), these same landmasses become very hot. The highest temperatures are found over subtropical continents like the Sahara and Gobi deserts, reaching 40°C. Meanwhile, Antarctica experiences its coldest period, with temperatures below -60°C.
- The most striking difference is how much more extreme the seasons are over land compared to the ocean.

- The equinox seasons, MAM (March-April-May) and SON (September-October-November), are much more moderate. Their temperature patterns look a lot like the annual average, acting as transitions between the winter and summer extremes.

Now, let's examine a single year of zonal winds:

In [9]:
```
filepath = 'ua_Amon_CESM2_01_12_1981_2012.nc' # choose the file
zonal_wind = xr.open_dataset(filepath)
zonal_wind
```

Out[9]: xarray.Dataset

► Dimensions: (**lon**: 288, bnds: 2, **lat**: 192, **time**: 12, **plev**: 19)

▼ Coordinates:

lon	(lon)	float64	0.0 1.25 2.5 ... 356.2 ...
lat	(lat)	float64	-90.0 -89.06 -88.12 .....
time	(time)	object	2010-01-15 12:00:00 ...
plev	(plev)	float64	1e+05 9.25e+04 ... 5...

▼ Data variables:

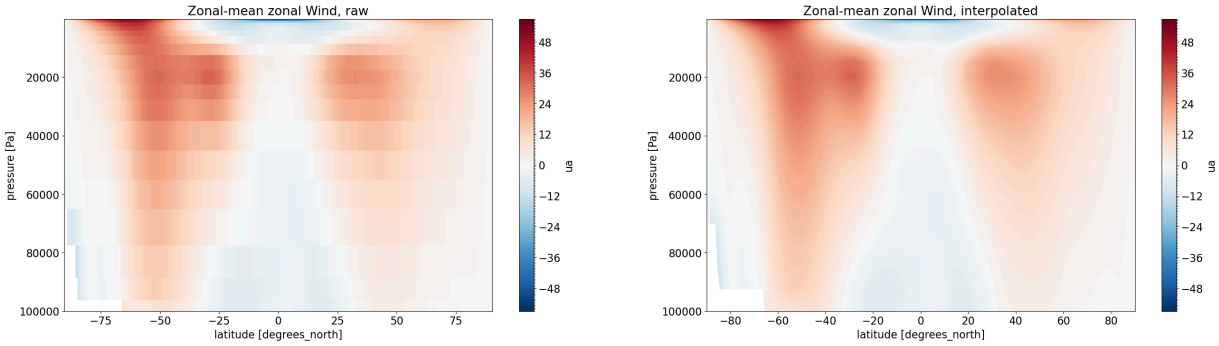
lon_bnds	(lon, bnds)	float64	...
lat_bnds	(lat, bnds)	float64	...
ua	(time, plev, lat, lon)	float32	...

► Attributes: (44)

You'll note the data in `zonal_wind` is now defined over coordinates `lat, lon, time, plev` and the data variable we want is `ua`. We can access this variable with `zonal_wind['ua']`.

Now let's plot annual-mean zonal-mean surface temperature, and describe it:

In [10]:
```
plot_data = zonal_wind['ua'].mean(['time','lon'])          # take the time m
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(32,8)) # creat a figure
axes_list = axes.flatten() # get a list of the axes

# we plot the data on each axis, with 200 levels in the colorbar, and y axis
# Note plot_data['plev'] and plot_data.plev both work, and we are going from
plot_data.plot(         ax=axes_list[0],levels=200,ylim=[plot_data.plev.max(
plot_data.plot.contourf(ax=axes_list[1],levels=200,ylim=[plot_data.plev.max(
_=axes_list[0].set_title('Zonal-mean zonal Wind, raw')      # title each plo
_=axes_list[1].set_title('Zonal-mean zonal Wind, interpolated') # title each
```



What patterns do you see in the data? At what level of the atmosphere are winds strongest? Why is this?

- Note red (positive) winds are westerlies (from the west, going east), while blue is the opposite
- Note also we've plotted both interpolated and uninterpolated data. It is good practice when creating plots to think about what data plot format implies about the resolution and amount of underlying data.

Answer:

- At the surface (bottom of the plot, ~100,000 Pa), the plot clearly shows the planet's major wind belts. You can see the **easterly winds** (blue), also known as the trade winds, in the tropics and strong **westerly winds** (red) in the mid-latitudes of both hemispheres. There are also weak polar easterlies near the South Pole.
- A major feature is that the winds generally become more westerly (more red) and increase in speed with altitude (as pressure decreases).
- The strongest winds are not at the surface but are concentrated in narrow bands in the **upper troposphere**, around 200-300 hPa (20,000-30,000 Pa). These cores of high-speed wind are the **jet streams**.
    - The reason for this structure is the strong temperature difference between the warm equator and the cold poles. The atmosphere's meridional (north-south) temperature gradient is directly related to the vertical shear of the zonal (east-west) wind. This relationship is known as thermal wind balance (**section 1.2.2**).

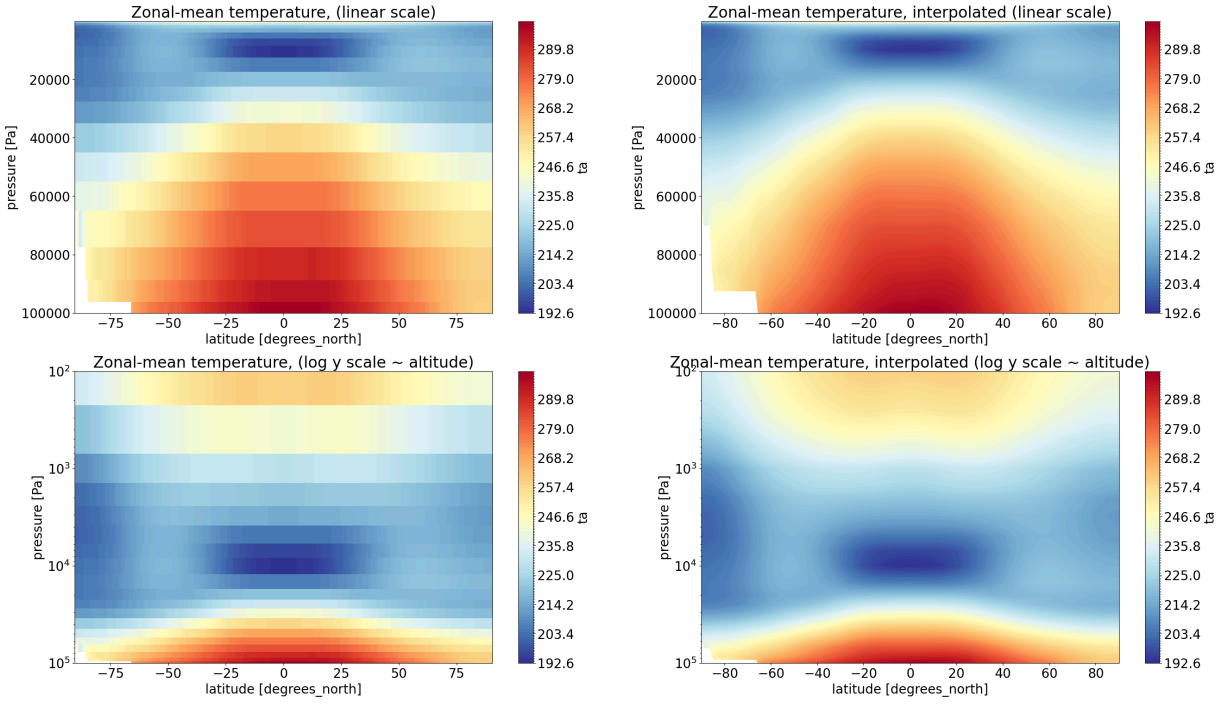
Now we will examine a similar dataset, but for temperature:

In [11]:
```
# Load the data
filepath = 'ta_Amon_CESM2_01_12_1981_2012.nc'
temperature = xr.open_dataset(filepath,decode_cf=True)
```

You'll note the data in `temperature` is stored in the same format as `zonal wind`. Now the data variable we want is `ta`. We can access this variable with `temperature['ta']`.

Now we will plot the data on both logarithmic nad linear vertical scales:

In [12]:
```
plt.rcParams.update({'font.size': 20})
plot_data = temperature['ta'].mean(['time','lon'])
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(32,18))
axes_list = axes.flatten()
plot_data.plot(          ax=axes_list[0],levels=200,cmap='RdYlBu_r'
plot_data.plot.contourf(ax=axes_list[1],levels=200,cmap='RdYlBu_r'
plot_data.plot(          ax=axes_list[2],levels=200,cmap='RdYlBu_r',yscale='l
plot_data.plot.contourf(ax=axes_list[3],levels=200,cmap='RdYlBu_r',yscale='l
_=axes_list[0].set_title('Zonal-mean temperature, (linear scale)')
_=axes_list[1].set_title('Zonal-mean temperature, interpolated (linear scale
_=axes_list[2].set_title('Zonal-mean temperature, (log y scale ~ altitude)')
_=axes_list[3].set_title('Zonal-mean temperature, interpolated (log y scale
```



Plotting on a logarithmic scale can help us get better context about the vertical structure of the atmosphere. Atmospheric pressure is related to altitude by the exponential decay formula:

$$P = P_{\text{sfc}}\ e^{-z/H}$$

for a scale height $H = RT/g \approx 8\ \mathrm{km}$. Thus, we can say for our logarithmic y axis:

$$y = \ln(P) = -z/H - P_{\mathrm{sfc}}$$

that is to say that the y axis is proportional to (not equal to) altitude, up to some additive constant i.e:

$$y = \ln(P) \propto z$$

What patterns do you see in the data? How does changing the vertical scale impact?

- Note that the resolution of the model decreases with altitude (z - It displays a much more unclear pattern in pressure given the exponential decay described above). Why do you think this is?
- How does the position of the winds we plotted before align with those of the temperatures we plotted now? (are the winds aligned with temperature maxima, minima, or gradients?

Answer:

- **Core Temperature Patterns:**
  - As seen before, the most obvious feature is that the atmosphere is warmest at the equator and cools as you move toward the poles. This temperature difference is the primary engine of the atmospheric circulation.
  - In the lower atmosphere (the troposphere, from the surface up to about 100-250 hPa), temperature decreases with altitude. This trend abruptly stops at the tropopause, which is the boundary between the troposphere and the stratosphere above it. The tropopause is highest over the tropics (around 100 hPa, ~17 km) and lower over the poles (~250 hPa, ~10 km). The coldest point in the entire lower atmosphere is at the tropical tropopause.
- **Impact of Changing the Vertical Scale:**
  - The linear pressure scale (top plots) makes it hard to see the structure of the upper atmosphere because pressure decreases exponentially with height. It squishes everything above the mid-troposphere into a narrow band at the top.
  - The logarithmic pressure scale (bottom plots) gives a much more intuitive view because log-pressure is proportional to altitude ($y = \ln(P) \propto z$). This scale stretches the vertical dimension, clearly

showing the structure of the tropopause and the stratosphere above it.

- **Model Resolution:** The model's vertical resolution is much finer near the surface and gets coarser at higher altitudes. This is a practical choice because about 80% of the atmosphere's mass and most of the weather we experience is in the troposphere (**section 1.1.4**). Climate models focus their computational power on this denser, more dynamic region.
- **Relationship Between Wind and Temperature:**
  - The strongest winds (the **jet streams**) are located directly adjacent to the strongest **horizontal temperature gradients**. You can see the cores of the westerly jets (from the previous plot) situated in the upper troposphere on the poleward side of the warmest air, where the temperature changes most rapidly between the equator and poles.
  - This is a direct result of **thermal wind balance**, which states that the vertical shear of the zonal wind is proportional to the meridional temperature gradient (**section 1.2.2**).

Now, we will calculate the global-mean temperature.

To do this we need to take the area-weighted mean. Because the earth is spherical, data points as high latitudes describe less area than data points at low latitudes. The proper latitude (ϕ) weighting is:

$$\text{weight} = \cos(\phi)$$

If we do not weight correctly, we may overreppresent high latitude areas that are cold but take up little area, decreasing the global-mean temperature. See http://xarray.pydata.org/en/stable/examples/area_weighted_temperature.html

Xarray can perform the weighting automatically:

In [13]:
```
weight = np.cos(np.deg2rad(temperature['lat']))           # convert degrees
temperature_weighted = temperature['ta'].weighted(weight) # calculate a weig
temperature_weighted # note this prints out a weighted object
```

Out[13]:
```
DataArrayWeighted with weights along dimensions: lat
```

We may also assume we can do this calculation ourselves. A naive implementation is the calculate the weights from the latitudes as before, and then multiply our data by the weights along the latitude axis ourselves.
Such an implementation may go as follows. We find the dimension of latitude in our temperature data array:

In [14]:
```
temperature['ta'].dims
```

Out[14]:
```
('time', 'plev', 'lat', 'lon')
```

We see from running the cell above that the right index is 2 (note python is zero indexed so `lat` is in the 2nd index position.
Now we:

- align our latitude data along the second dimension
- multiply our data by the weights (note we normalize the weights by their sum so that $\sum(\text{normalized weights}) = 1$ and the global mean magnitude is correct.

In [15]:
```
lat_data = temperature['lat'].data      # Take the underlying data from the
lat_data = lat_data.reshape(1,1,-1,1)   # The lat dimension is 2, so reshape
weight2  = np.cos(np.deg2rad(lat_data)) # Calculate our weights...
weight2  = weight2/np.sum(weight2)      # ... and normalize them
temperature_weighted_2 = temperature['ta'] * weight2 # apply the weights to
```

This implementation, however, despite its simplicity, is incorrect!

Note in the previous 2 plots the white area at the surface in the southern hemisphere. This is missing data. It occurs because the surface in Antarctica is so elevated that the pressure at the surface there is higher than that at the bottom of our y/pressure axis. This is only visible at low latitudes where antartica spans all longitudes, but high elevation regions across the globe such as the Himalayas would display a similar issue!

Thus, our normalization is a little bit off, we should only consider weights for data we actually have. It seems we'll have to be a bit more precise with our calculation...

A more thorough implementation may go as follows:

- align our data along the correct (2nd) dimension (already done!)
- broadcast it to match the size of our tempearture data, (i.e. expand by repetition so it matches the size of the temperature data)
- drop weights in the areas where temperature data is missing
- normalize along the latitude dimension (2) based on the weights that remain

This is implemented below. I encourage you to pull out any variable and examine it to help you understand what is going on. For example, one might examine `temperature['lat'].shape, lat_data.shape, lat_data_grid.shape` to see their respective shapes.

In [16]:
```
lat_data_grid = np.broadcast_to(lat_data,temperature['ta'].shape) # use nump
valid    = ~np.isnan(temperature['ta'])                           # valid is
```

```
weights3  = np.cos(np.deg2rad(lat_data_grid))                    # take th
weights3 = weights3 * valid                                     # multiply
weights3  = weights3/np.sum(weights3,axis=2)                       # sum a
temperature_weighted_3 = temperature['ta'] * weights3            # apply t
```

Now we can calculate some global means. We have to be a little careful with our weights. The definition of a weighted mean is:

$$\langle x \rangle_w = \frac{\sum w_i x_i}{\sum w_i} = \sum x_i \frac{w_i}{\sum w_i}$$

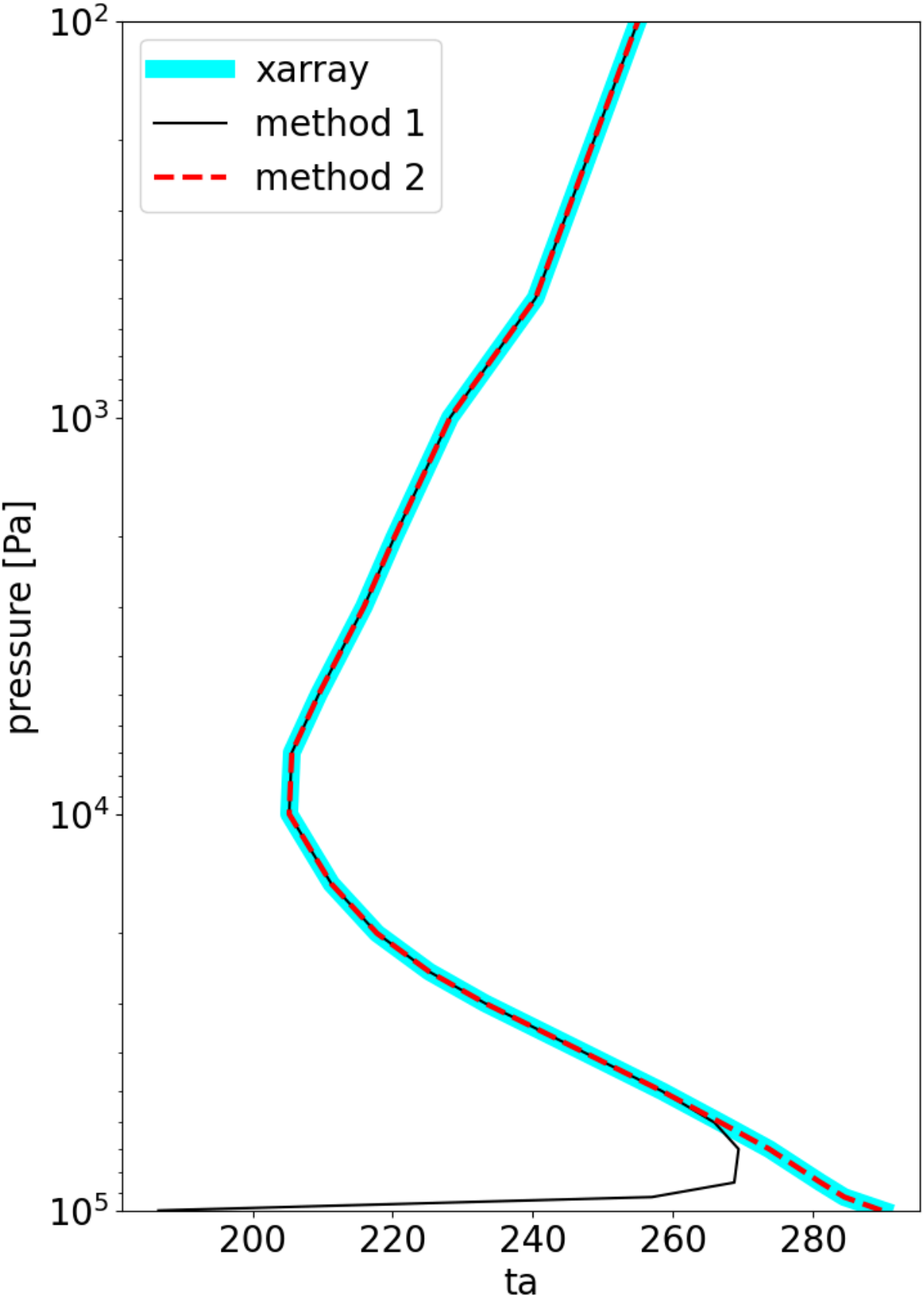
and we have calculated by multiplication:

$$\langle x \rangle_w = x_i \frac{w_i}{\sum w_i}$$

So the only remaining operation is a sum.

We calculate and plot our 3 methods of doing this calculation below:

In [17]:
```
fig = plt.figure(figsize=(8, 12)) # assign a vertically tall figure
ax  = plt.axes()                  # assign axes to the figure
plot_data     = temperature_weighted.mean(['lat','lon','time']).transpose()
plot_data_2 = temperature_weighted_2.sum(['lat']).mean(['lon','time']).trans
plot_data_3 = temperature_weighted_3.sum(['lat']).mean(['lon','time']).trans

plot_data.plot(  ax=ax, y='plev',yscale='log',ylim=[plot_data.plev.max()  ,p
plot_data_2.plot(ax=ax, y='plev',yscale='log',ylim=[plot_data_2.plev.max(),p
plot_data_3.plot(ax=ax, y='plev',yscale='log',ylim=[plot_data_3.plev.max(),p
_ = ax.legend() # you see the problems at the bottom from some missing data
```



Thankfully, we see our second method matches the native xarray method! The first, however, is incorrect, primarily because it assigned positive weights to missing data decreasing the total value of our averages.
This can teach us a few important lessons:

- We should always be very careful with our calculations and be aware of what our data actually contains to avoid careless errors such as the one we might have made here
- **Perhaps more importantly, it is generally better to use trusted official libraries and packages to perform calculations rather than write your own.**

What interesting features do you see in the correct profiles? Where is the tropopause?

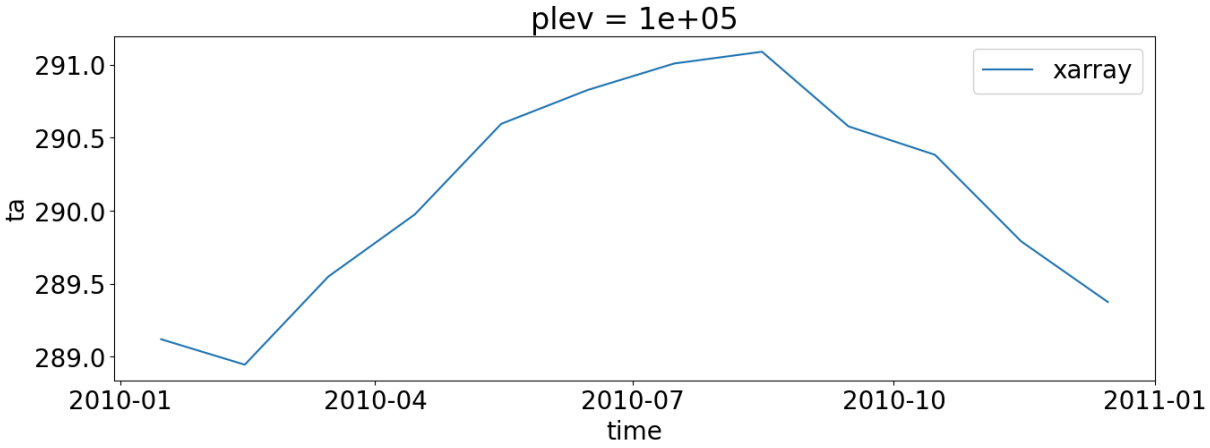
Answer:

- **Troposphere:** The profiles clearly show that in the lowest layer of the atmosphere (troposphere), temperature consistently decreases with altitude (as pressure decreases). This rate of cooling with height is called the temperature lapse rate (**section 1.1.4**). The temperature at the surface is around 289 K, which we know from class is the global- and annual-mean surface air temperature.
- **Tropopause:** The most distinct feature is the sharp kink in the profile where the temperature stops decreasing. This inflection point is the tropopause, the boundary between the troposphere and the stratosphere. It's located on the plot around **20,000 Pa**, which is where the temperature reaches its minimum value of roughly 215 K.
- **Stratosphere:** Above the tropopause, the temperature trend flips. In the stratosphere, the temperature begins to increase with altitude. This warming occurs because ozone in this layer absorbs solar radiation, heating the surrounding air (**section 1.1.4**).

Now let's examine the annual temperature cycle at the lowest model level:

We use xarray's native weighting and then take a mean over the `lat` and `lon` dimensions. Then we use index selection `isel` to select the lowest elevation pressure level.

In [18]:
```
fit = plt.figure(figsize=(15, 5)) # create a figure
ax  = plt.axes()                  # attach axes
plot_data     = temperature_weighted.mean(['lat','lon']).isel(plev=0).transp
plot_data.plot(  ax=ax, label='xarray') # plot the data
_ = ax.legend()
```



Is there an annual cycle in temperature, and if so, why? (Hint: How large is the variability in this plot compared the annual cycle of temperatures in any individual location?)

Answer:

Yes, there is a small but clear annual cycle in the global-mean temperature, even though the temperature variation is only about 2 K.

This cycle exists because of the uneven distribution of land and oceans between the two hemispheres. The Northern Hemisphere has much more landmass than the Southern Hemisphere. As **section 1.1.1** explains, continents heat up and cool down far more dramatically with the seasons than oceans do, an effect called **"continentality."**

Because of this, the large landmasses of the Northern Hemisphere experience a much hotter summer and a much colder winter compared to the mostly-ocean Southern Hemisphere. When you average the temperature across the entire globe, the extreme seasons of the Northern Hemisphere dominate the average, causing the global-mean temperature to peak during the northern summer (JJA) and reach a minimum during the northern winter (DJF).

As the hint suggests, this global cycle is tiny compared to the temperature swings in any single location. While the global average only varies by ~2 K, the seasonal plots we saw before show that a place like Siberia can experience a temperature range of more than 50 K between its hot summer and frigid winter. The global average is so muted because the summer in one hemisphere is happening at the same time as the winter in the other, largely canceling each other out.

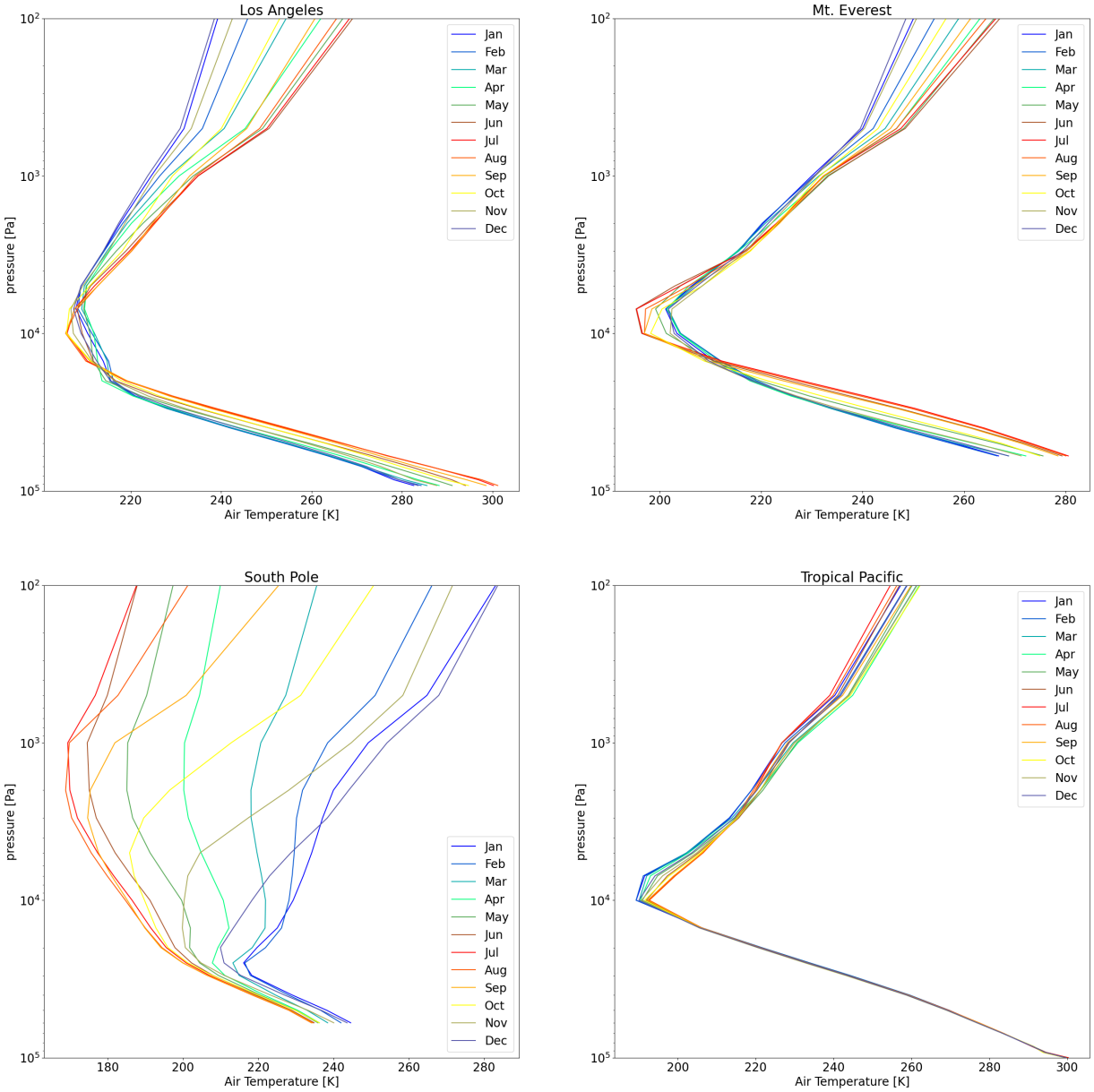
Now, let's return to our vertical profiles and see what the temperature looks like in a selection of locations around the world.

First, we prepare our data by selecting data nearest to the coordinates of our locations:

In [19]:
```
# get the coordinates of some locations
coords = {'Los Angeles'      : [34.05349 , 241.75468], # [ lat, lon]
          'Mt. Everest'      : [27.988056, 86.92528 ],
          'South Pole'       : [-90.0    , 0        ],
          'Tropical Pacific' : [0        , 180      ]
         }
# reate a dictionary of data and fill it with data
data ={}
for location,coordinates in coords.items(): # loop over the name,value pairs
    data[location] = temperature['ta'].sel(lat=coordinates[0],lon=coordinate

# Create a colormap that cycles between cold to warm colors
from matplotlib.colors import ListedColormap, LinearSegmentedColormap
colors = cmap = LinearSegmentedColormap.from_list("mycmap", [[0,0,1],[0,1,.5
colors = cmap(np.linspace(0, 1, 13)[0:12])

months = np.array(temperature.time.dt.month) # get month numbers from our da
month_names = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','
```

Now that we've prepared the data, we can plot it. We'll plot it by month so we can see the annual cycle:

In [20]:
```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(32,32)) # prepare our 2x
axes_list = axes.flatten()             # get a list of axes from the flatter
plt.rcParams.update({'font.size': 20}) # set font size

for i,location in enumerate(data.keys()): # iterate over our data and its ir
    for month in months:
        plot_data = data[location].isel(time=month-1) # months are not zero
        plot_data.plot(ax= axes_list[i], y='plev',yscale='log',ylim=[plot_da
    axes_list[i].set_title(location) # title iwth the location
    axes_list[i].legend(fontsize=20) # add a legend to show the months
```



What sort of patterns do you see in the data?

It is okay to be broad or vague here, we don't expect you to necessarily know what to look for! Some ideas might include:

- How does the surface temperature compare with your expectation of cold and warm regions across the globe?
- How large is the annual cycle? At the surface? Aloft? What months are warm? What months are cold?
- If you have any insight into why these variations might occur, feel free to provide them, if not that's okay! We'll go over some of this in recitation and office hours
- Is the rate of change of temperature with height a constant? (This is a log pressure scale, i.e. proportional to altitude, so in the troposphere the answer should be yes!)

- If you're particularly observant, you'll note a slight increase in the lapse rate $\left(\frac{\partial T}{\partial z}\right)$ in the upper vs lower troposphere.
  This is particularly visible in the tropical atmosphere. Why might this exist?
- How does the location of the tropopause vary with location? How does this compare with the zonal-mean cross section maps of temperature we made earlier?

Answer:

- **Los Angeles (Mid-latitude Coast):**
  - The surface temperatures are warm, as expected, with a mild annual cycle of about 10 K. The warmest months are in late summer (August-October), and the coolest are in winter (January-February). This relatively small temperature swing is due to the moderating effect of the nearby ocean, which has a high thermal inertia.
  - The tropopause is located around 200 hPa, which is a typical altitude for the mid-latitudes, consistent with the zonal-mean plots.
- **Mt. Everest (High Altitude):**
  - The profile starts at a much lower pressure (~60,000 Pa or 600 hPa) because of the mountain's extreme elevation. The "surface" temperature at this altitude is very cold year-round, which makes sense for the highest point on Earth.
  - There's a clear seasonal cycle, with the Northern Hemisphere summer (JJA) being warmest and winter (DJF) being coldest.
- **South Pole (Polar Region):**
  - This is by far the coldest location, with a massive annual temperature cycle at the surface. The seasons are flipped, with the warmest month being January (austral summer).
  - A strong temperature inversion occurs during the polar winter (JJA), where the temperature actually increases with height just above the surface before starting to cool again.
- **Tropical Pacific (Equatorial Ocean):**
  - The surface temperature is very warm (around 298 K) and incredibly stable, with an almost non-existent annual cycle.
  - The tropopause is very high (around 100 hPa or ~17 km) and extremely cold (~195 K). This matches **section 1.1.4**'s description of the tropical tropopause acting as a cold trap that freezes out water vapor.

- As one of the bullet points hint, the lapse rate (rate of cooling with height) is not perfectly constant. It's slightly steeper in the lower troposphere and becomes less steep in the mid-to-upper troposphere.